DAA / ASSIGNMENT 1

Запуск, сдача и защита проекта

Практическое руководство к проекту
Divide-and-Conquer Algorithm Analysis

В комплекте: четыре алгоритма на Java, 24 692 успешные проверки, 672 строки измерений, четыре графика, README на английском и последовательная история Git.

1. Открой правильную папку

Полностью распакуй ZIP. Открой папку **assignment1-divide-and-conquer**: внутри должны находиться pom.xml, README.md, src, tests, docs и results. Команды ниже выполняются из этой папки.

2. Проверь Java

```
java -version
```

Нужен **JDK 17 или новее**. JDK включает компилятор; одного старого JRE недостаточно. В IntelliJ IDEA можно открыть pom.xml и выбрать JDK 17.

3. Запусти на Windows

```
powershell -ExecutionPolicy Bypass -File scripts/run.ps1 demo
powershell -ExecutionPolicy Bypass -File scripts/run.ps1 test
```

Первая команда показывает работу алгоритмов. Вторая должна закончиться строкой:

```
ALL TESTS PASSED: 24692 checks
```

На Linux, macOS или в Git Bash используй **bash scripts/run.sh demo** и **bash scripts/run.sh test**. Прямой Java/Bash-запуск проверен; Maven и PowerShell в среде измерений отсутствовали.

Как сдать через GitHub

По условию сдаётся **ссылка на репозиторий**. Сам архив или PDF не заменяет эту ссылку. Публикация в твоём аккаунте пока не выполнена.

1. Создай пустой репозиторий

На GitHub создай assignment1-divide-and-conquer. Не добавляй новый README, лицензию или .gitignore при создании: локальный проект уже содержит нужные файлы. Выбери видимость, при которой преподаватель получит доступ.

2. Перенеси проект вместе с историей

Нужен установленный Git. Замени YOUR_USERNAME своим логином и выполни команды из папки проекта:

```
git status
git log --oneline --reverse
git remote add origin https://github.com/YOUR_USERNAME/assignment1-divide-and-conquer.git
git push -u origin main
git push origin ci/automated-tests
git push origin --tags
```

При запросе авторизации войди через браузер или настроенный менеджер учётных данных. Не требуется передавать пароль в переписку.

3. Проверь страницу перед отправкой

На GitHub должны открываться исходники, тесты, README, графики и CSV. Проверь ветки main и ci/automated-tests, коммиты и результат Actions. Отчёт сдаётся внутри README.md. Затем отправь в Moodle адрес страницы репозитория.

Если история не распозналась

Сначала проверь рабочую папку и полноту распаковки. В архиве есть резервный **assignment1-history.bundle**. Из папки с этим файлом можно восстановить репозиторий:

```
git clone assignment1-history.bundle assignment1-restored
```

Дальнейшие команды подключения origin и отправки приведены в docs/SUBMISSION_RU.md. Загрузка через Upload files не переносит существующую историю Git.

Коммиты отражают реальные этапы подготовки в текущей сессии; даты и авторство не изменялись. Для своих дальнейших изменений используй собственные имя и почту Git.

Четыре алгоритма: основная идея

MergeSort

Делим массив пополам, сортируем половины и сливаем их. На каждом уровне суммарная работа линейная, уровней порядка log n. Один вспомогательный массив переиспользуется. Части длиной не больше 24 обрабатываются сортировкой вставками.

Что показать: создание auxiliary один раз, условие INSERTION_CUTOFF и цикл линейного слияния в MergeSorter.java.

QuickSort

Случайный pivot делит массив на области меньше, равно и больше него. Равная область уже готова. Для меньшей части выполняется рекурсивный вызов, большая обрабатывается циклом. Повторы не создают цепочку бесполезных разбиений.

Главная мысль: стек O(log n) даже при плохих pivot, но худшее время остаётся $O(n^2)$. Маленький стек и небольшое число операций - разные свойства.

Deterministic Select

Ищем элемент ранга k, не сортируя весь массив. Сортируем группы по пять, собираем их медианы и находим медиану медиан. После разбиения продолжаем только в части с нужным k. Индексация начинается с нуля.

Пример: для [9, 1, 7, 3, 5] и k=2 ответ 5. В экспериментах используется k=n/2 - верхний центральный элемент при чётной длине.

Closest Pair

Сортируем точки по x, решаем левую и правую половины. Проверяем полосу около границы, где может лежать более близкая пересекающая пара. Точки в полосе идут по y; достаточно не более семи следующих соседей.

Ключевая деталь: порядок по y поддерживается слиянием. Сортировка полосы заново на каждом уровне дала бы $O(n \log^2 n)$.

Сложность и её объяснение

Алгоритм	Время	Доп. память
MergeSort	Θ(n log n)	O(n)
QuickSort	Ожидаемое Θ(n log n), худшее Θ(n²)	O(log n)
Select	Худшее Θ(n)	O(log n)
Closest Pair	Θ(n log n)	O(n)

Master Theorem: MergeSort и Closest Pair

```
T(n) = 2T(n/2) + Theta(n)
a = 2, b = 2, log_b(a) = 1
```

Нерекурсивная работа совпадает по порядку с n в степени log_b(a). Это второй случай Master Theorem: результат Θ(n log n). Для Closest Pair первоначальная сортировка по x тоже укладывается в O(n log n).

Почему Select имеет линейную гарантию

Примерно половина медиан групп находится по каждую сторону pivot. Каждая подходящая полная группа даёт три значения с нужной стороны. Поэтому можно отбросить около 3n/10 значений, с постоянной поправкой из-за округления.

```
T(n) <= T(n/5) + T(7n/10) + O(n)
1/5 + 7/10 = 0.9 < 1
```

Обычная Master Theorem напрямую не применяется: подзадачи разных размеров. В объяснении Akra-Bazzi показатель p удовлетворяет (1/5)^p + (7/10)^p = 1. Он лежит между 0 и 1; линейная работа вне рекурсии даёт итог O(n). Полный вывод приведён в README.

Почему QuickSort может оставаться квадратичным

При постоянно крайних pivot остаётся задача размера n-1: T(n)=T(n-1)+Θ(n). Сумма n+(n-1)+... даёт Θ(n²). Цикл для большой части экономит стек, но не убирает эту работу.

Вопросы на защите

Как проверялась правильность?
Сортировки и Select сравниваются с Arrays.sort. Closest Pair сравнивается с полным перебором для небольших наборов, включая n=2000. Большой набор проверяется также на известном расстоянии.

Зачем System.nanoTime()?
Для измерения прошедшего времени: берём разницу значений до и после алгоритма. Генерация входа и эталонная проверка не входят в этот интервал.

Почему есть прогрев JVM?
Виртуальная машина компилирует часто используемые участки во время работы. Без прогрева первые запуски могут быть несопоставимы с последующими.

Почему семь повторов и медиана?
Один запуск может замедлиться из-за GC или других процессов. Медиана уменьшает влияние единичного выброса; исходные семь результатов сохранены в CSV.

Что означает seed?
Он фиксирует последовательность псевдослучайных чисел. Данные и pivot воспроизводятся; время может отличаться.

Что происходит с пустым входом?
Сортировки завершаются без действий. Select отклоняет недопустимый k. Closest Pair сообщает об отсутствии пары.

Что с повторяющимися значениями?
QuickSort и Select используют отдельную область равных pivot. Совпадающие точки разрешены: их расстояние равно нулю.

Что можно улучшить дальше?
Добавить отдельные процессы JVM для измерений, подобрать cutoff экспериментально и сравнить версии со счётчиками и без них. Это планы, а не выполненные части.

Подробные ответы и короткое вступление на английском находятся в **docs/DEFENSE_RU.md**. Перед защитой обязательно проследи хотя бы один пример вручную для каждого алгоритма.

Результаты и финальная проверка

Для случайного входа из 300 000 элементов/точек в сохранённой среде получены следующие медианы семи запусков:

Алгоритм	Время, мс	Макс. глубина
MergeSort	34.211	15
QuickSort	49.707	12
Select	14.118	18
ClosestPair	245.822	18

На 2000 случайных точек: быстрый поиск около 0,702 мс, полный перебор около 35,550 мс. Наблюдаемое ускорение - примерно 50,7 раза. Это результат конкретного запуска, а не обещание такой скорости на любом компьютере.

Что уже покрыто по критериям

Раздел	Вес	Где смотреть
Алгоритмы	40%	src/daa
Реализация и эксперименты	20%	Experiment.java, results
Тестирование	10%	AlgorithmTests.java, tests.txt
README и изображения	15%	README.md, docs
Структура и история Git	15%	Локальная история готова; нужна публикация на GitHub

Перед сдачей

1. Запусти demo и test на своём компьютере.
2. Сделай собственные скриншоты терминала, если преподавателю нужны именно снимки экрана. В комплекте - изображения реального сохранённого вывода.
3. Перечитай Reflection и поправь его под свой реальный опыт.
4. Проверь графики и доступ к репозиторию.
5. Отправь ссылку GitHub в LMS.

Оценку выставляет преподаватель. Этот комплект закрывает техническую подготовку; итог зависит также от публикации, требований курса и того, насколько уверенно ты объяснишь решение.